ASSIGNMENT 1 - TABULAR METHODS EXPERIMENTATION

OPTION A: OPTUNA HYPERPARAMETER ANALYSIS

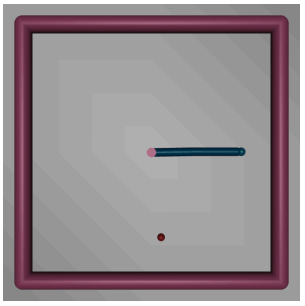


SUBJECT: Reinforcement Learning

GROUP MEMBERS: Lucia Diez and Iria Picó
GMAILS: lucia.diez@opendeusto.es iria.pico@opendeusto.es

1. Environment selected and motivation

The main objective of this assignment was to compare three tabular Reinforcement Learning algorithms: **SARSA, Expected SARSA and Q-Learning**. We used Optuna to explore different hyperparameter configurations and analyze how they affect the learning process.



We selected Reacher-v5, a Gymnasium environment based on the MuJoCo physics simulator. This environment consists of a robotic arm with two connected joints.
→ **OBJECTIVE**: move the end of the arm as close as possible to a target position applying force joints.

We chose this environment because we were interested in working with a robotic control problem. We also wanted an environment that was **challenging** enough to study limitations of tabular methods. Unlike simpler environments with discrete state and actions, Reacher used continuous values:

Characteristic	Description
Observation space	10 continuous values
Action space	2 continuous control values
Episode length	50 steps
Objective	Move the robotic arm close to a target
Reward	Penalizes distance from the target and control effort

The **reward function** combines two components:

- Distance penalty: the further the arm is from the target → the more **negative**
- Control penalty: applying larger forces also reduces a penalty

Therefore, a higher reward (closer to zero) indicates better performance according to the environment's reward function.
We considered Reacher an interesting choice because it allowed us to investigate whether tabular algorithms could learn useful behavior in a **continuous robotic** control task, even if they could **not solve** it completely.

Rather than focusing only on solving the environment, our goal was to carry out a systematic experiment. We wanted to **understand** how different parameters influence performance, whether the three algorithms behave differently under similar conditions, and how reliable the results were.

To do this, we first explored the environment and established some initial baselines. Then, we tested the three algorithms, optimized their hyperparameters with Optuna, and selected the best configurations for longer training runs.

→ An important part of our approach was not only to look at the final rewards, but also to **understand** why some configurations performed better than others.

ENVIRONMENT DISCRETIZATION

Since **SARSA, Expected SARSA and Q-Learning** use Q-tables, we applied the DiscretizationWarppers provided in class to convert the continuous spaces into discrete ones.

→ We used 3 bins for each observation dimension, obtaining **59,049 discrete states**

Initially, we also used 3 bins for the actions, obtaining 9 possible actions. However, during our preliminary experiments, we discovered that one of these actions represented zero force [0, 0].

→ Our final **Q-table** contained 59,049 states and 16 actions (**944,784 Q-values**)

This helped us understand that discretization is not just a **technical step**: it directly affects the actions that the agent can learn and the performance it can achieve.

2. Hyperparameters Explored and Their Ranges

Once we confirmed that three algorithms could train in Reacher, we used **Optuna** to explore different hyperparameter configurations and identify which ones produced better results.

We performed 50 trials for each algorithm, for a total of **150 trials**. In each trial, the agent was trained for 30,000 steps and evaluated over 20 episodes.

The objective was to **maximize** the mean episode reward. Since Reacher mainly produces negative rewards, higher values (closer to zero) indicate better performance. We explored the following hyperparameters:

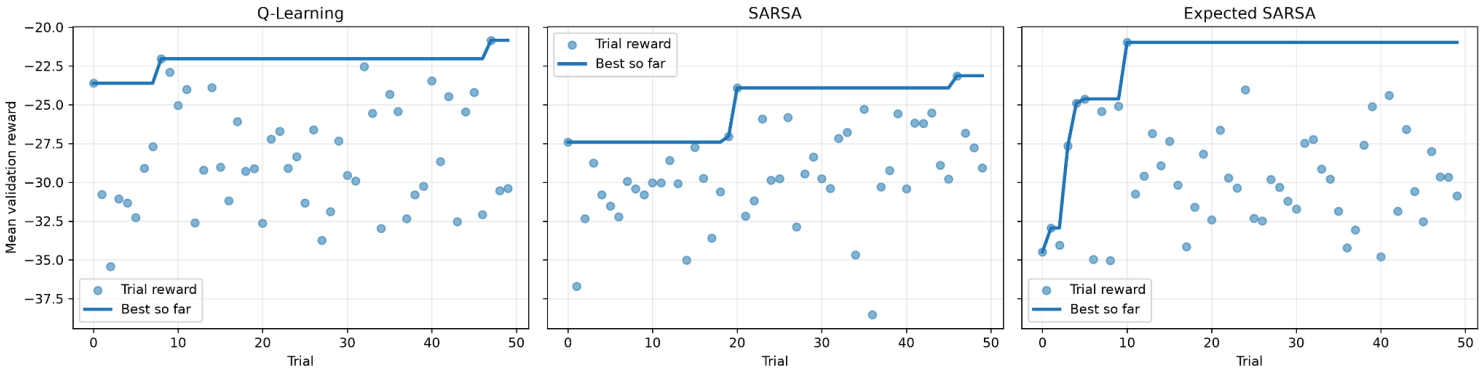
Hyperparameter	Range	Description
Learning rate (α)	0.01–0.50	Controls how much the Q-values change during learning
Discount rate (γ)	0.80–0.999	Determines the importance of future rewards
Epsilon (ε)	0.05–0.50	Controls the balance between exploration and exploitation
Learning rate decay	0.90–1.00	Progressively reduces the learning rate

The learning rate was explored on a logarithmic scale, and the decay was applied every 100 episodes.

We used **Optuna's TPE sampler**, keeping the training budget and evaluation procedure the same for all the algorithms. This allowed us to compare the effect of different configurations under similar conditions.

3. Optuna Graph Results and Anlysis

Figure 1. Optuna Optimization History



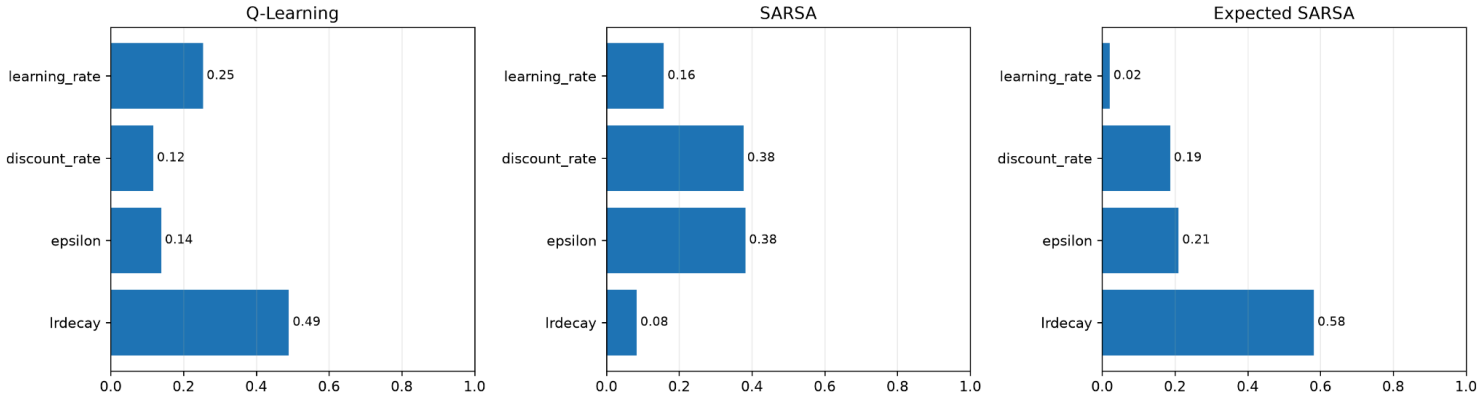
This graph shows the reward obtained in each trial and the best value found so far. One of the most interesting observations is that the three algorithms improved at different moments of the optimization process.

Expected SARSA found its best configuration quite early, around **trial 10**. After that, Optuna continued exploring, but no other combination improved that result. On the other hand, **Q-Learning and SARSA** found better configurations much later, close to the final trials.

We also observed considerable variation between trials. Even after finding a good configuration, other combinations produced much worse rewards.

This showed us that hyperparameters selection has an important influence on performance, and that testing only one configuration would not be enough to understand how each algorithm behaves.

Figure 2. Hyperparameter Importance Comparison

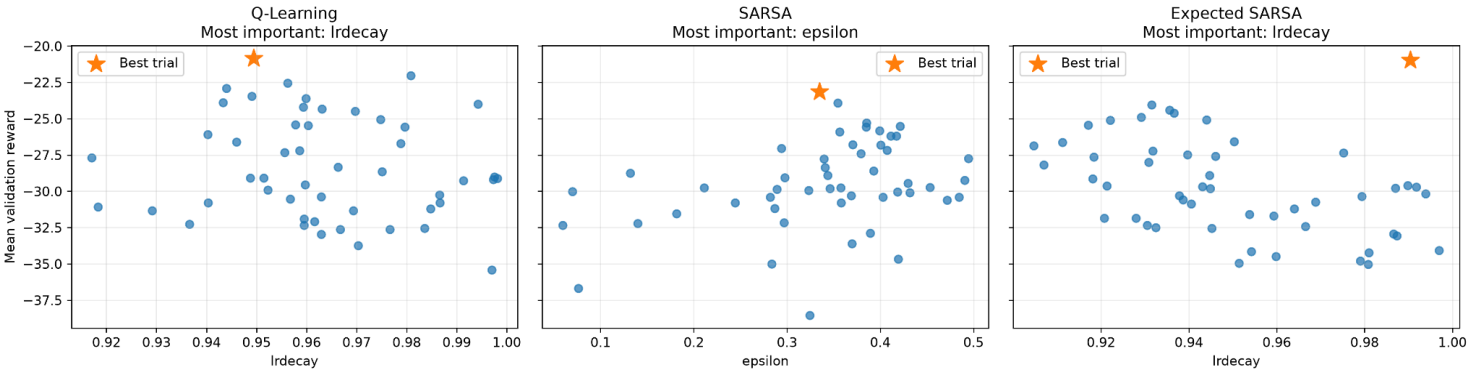


This graph compares the estimated importance of the four hyperparameters.
The results show some interesting differences:

- **Q-Learning**: The most important parameter was learning rate decay (0.49), followed by learning rate (0.25)
- **SARSA**: Epsilon and discount rate were the most influential, both with an importance of approximately 0.38
- **Expected SARSA**: Learning rate decay was again the most important (0.58), while learning rate had a very low estimated importance (0.02)

We found this **particularly interesting** because the same hyperparameters did not have the same importance for all the algorithms, even though they were trained in the same environment.
This suggests that each algorithm may benefit from a different learning strategy, rather than using the same hyperparameter for all of them.

Figure 3. Hyperparameters and Performance



To investigate the previous results further, we analyzed the relationship between the most important hyperparameter of each algorithm and the obtained reward.
The results show that similar parameter values can produce quite **different rewards**. For example, in Q-Learning, configurations with similar learning rate decay values sometimes obtained very different results.
→ This indicates that performance does not depend on one parameter alone, but also on its **combination** with the others.

4. Best Hyperparameter Configurations

After completing the 50 trials for each algorithm, we selected the combination with the highest mean validation reward. The best configurations:

Hyperparameter	Q-Learning	SARSA	Expected SARSA
Learning rate	0.01719	0.01219	0.31777
Discount rate	0.94665	0.97039	0.82137

Epsilon	0.41938	0.33466	0.16611
Learning rate decay	0.94939	0.95996	0.99042
Best Mean Reward	**–20.84**	**–23.13**	**–20.97**

Q-Learning achieved the **highest validation** reward (-20.84), although Expected Sarsa obtained a very similar result (-20.97). Sarsa achieved a slightly low reward (-23.13).
Another interesting observation is that the best hyperparameter configurations were quite different. For example, **Expected Sarsa** used a much higher learning rate and a lower discount rate than the other ones. This supports our previous observation that different algorithms can benefit from different hyperparameter configurations.
However, these results correspond to the Optuna validation phase, so they do not necessarily mean that Q-Learning is the best algorithm overall. For this reason, we used the three selected configurations for **longer training runs** of 200,000 steps, followed by evaluations with **different seeds**.

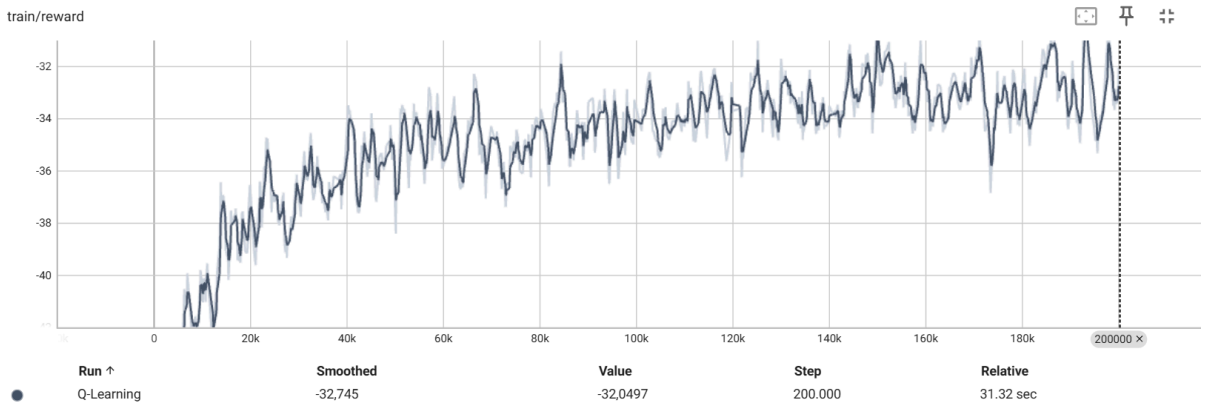
5. Long training and TensorBoard

Each algorithm was retrained for 200,000 steps with the best configuration found by Optuna, longer than the 30,000 step trials used in the search. In the training, we used an epsilon-greedy policy with the tuned epsilon, the learning rate decayed every 100 episodes (n_episodes_decay=100), with the TensorBoard logging every 20 episodes.

Algorithm	learning_rate	discount_rate	epsilon	lrdecay
Q-Learning	0.01719	0.94665	0.41938	0.94939
SARSA	0.01219	0.97039	0.33466	0.95996
Expected SARSA	0.3178	0.8214	0.1661	0.9904

The following graphs show the training reward during the 200,000 steps of each algorithm, recorded by TensorBoard every 20 episodes. We describe each curve first, and then compare the three together.

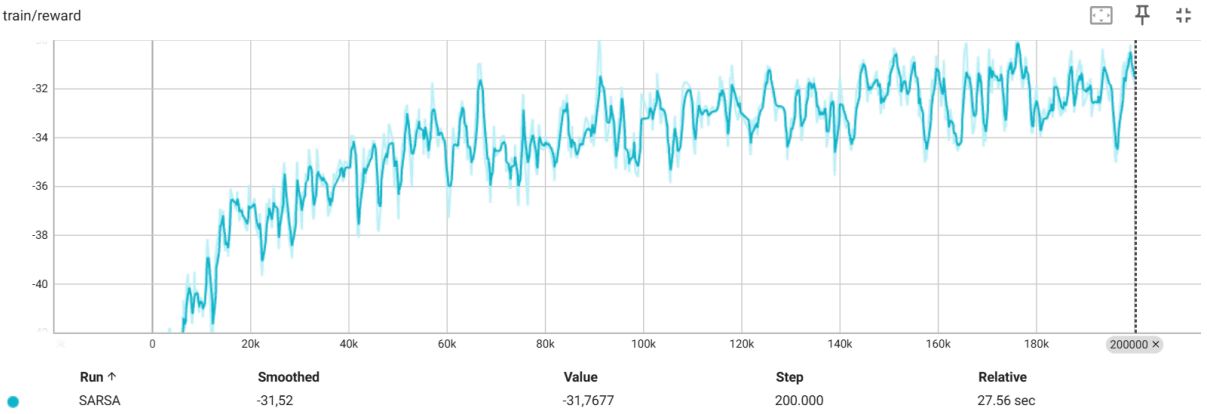
Q-Learning



In the Q-Learning training graph we can see how the reward improves quickly at the beginning: going from approximately -41 and in the first 20,000 steps goes up to -37. After this point more or less, the improvement is slower, between steps 100,000 and 200,000 the curve stays almost flat around -33, with a final smoothed value of -32.7.

The curve is noisy, with oscillations of 2 or 3 points between an episode and the next. This is partly due to the high value of the epsilon (0.42), meaning that the agent acts randomly in a large part of the steps. The curve flattens at the same time as the learning rate drops. The learning rate is multiplied by 0.949 every 100 episodes, so by the end of training it is much smaller and the agent barely changes its Q-table.

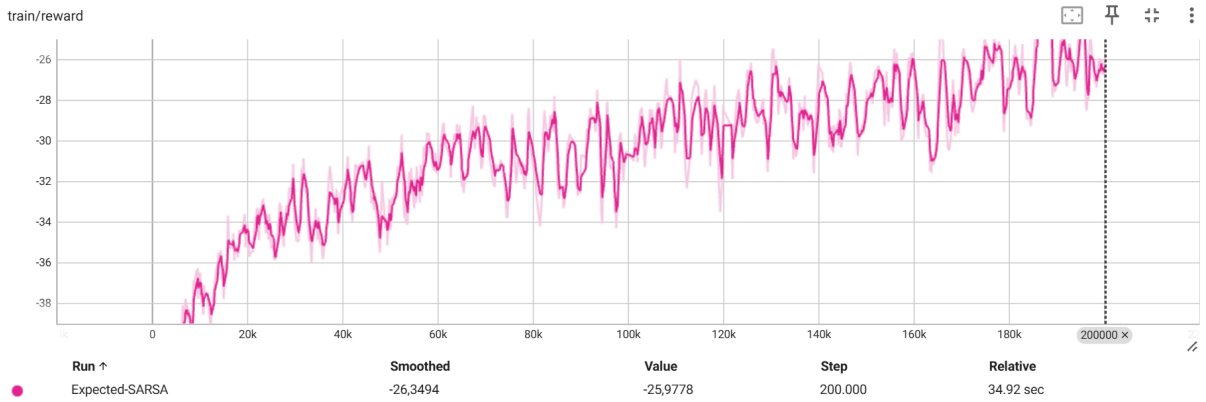
SARSA



Here, we can see how the reward also improves fast at first: as Q-Learning it starts at approximately −41 and rises to about −37 in the first 20,000 steps. After that, it keeps improving more slowly but steadily, and around step 100,000 it is close to −34. In the second half of training the improvement is very small: the curve moves between −35 and −31, with a final smoothed value of −31.5.

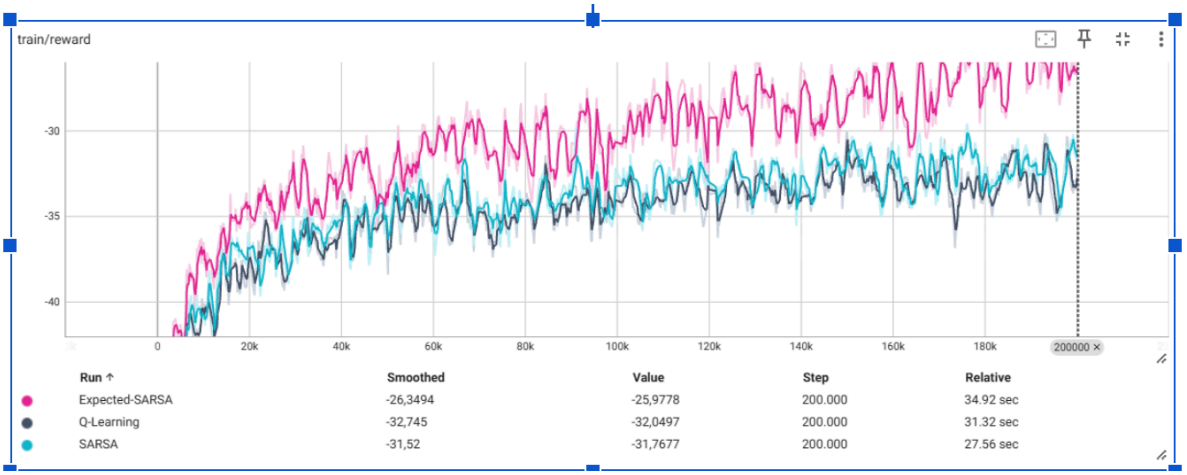
The curve is also noisy due to the epsilon of around 0.34. SARSA ends slightly above Q-Learning (−31.5 compared with −32.7), but the difference is small compared with the noise of both curves.

Expected SARSA



In the Expected SARSA graph, the reward improves fast at the beginning. The curve starts at around −39 and gets to approximately −34 in the first 30,000 steps. In this case, the curve does not stop improving like the other two. It keeps going up until the end: it is about −30 around step 100,000 and about −26 at the last steps, with a final smoothed value of −26.3. A possible reason is that this configuration uses a higher learning rate (0.32) that decreases very little, so the agent continues updating the Q-table during the whole training.

The curve is quite noisy, with oscillations of 3 or 4 points between consecutive points. Expected SARSA gets a clearly higher training reward than the other two algorithms. However, this reward includes the exploration, and this configuration explores less (epsilon 0.17), so part of the difference could be caused by that and not only by better learning.



This compares the three algorithms in the same graph. Expected SARSA is above the other two from the first steps, and the difference becomes bigger during the training: it is around 4 points at step 100,000 and around 6 points at the end.

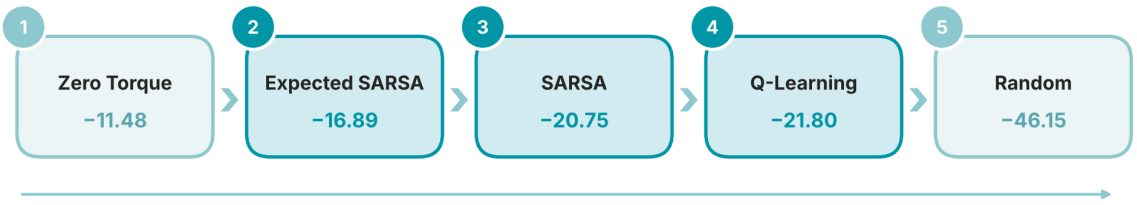
Q-Learning and SARSA are very similar during the whole training, and their curves overlap most of the time. SARSA finishes a bit higher than Q-Learning, but this difference is small if we compare it with the noise of the curves.
In addition, we should take into account that this is the training reward, and it includes the random actions of the exploration. Expected SARSA uses the lowest epsilon (0.17), so part of its advantage could come from acting randomly less often. In addition, these curves come from only one training run per algorithm.

6. Final Evaluation

All three algorithms are much better than the random policy, but none of them is better than the Zero Torque policy, which never applies force. We evaluated every policy with the greedy policy on the same 50 episodes (evaluation seeds 5000–5049).

Method	Mean reward (50 episodes)	Note
Zero Torque	−11.48	Reference: constant action (0, 0)
Expected SARSA	−16.89	Mean of 3 training seeds
SARSA	−20.75	Mean of 3 training seeds
Q-Learning	−21.80	Mean of 3 training seeds
Random	−46.15	Reference: uniformly random actions

Looking at the table, Zero Torque gets the best reward, followed by Expected SARSA, SARSA and Q-Learning. Random is clearly the worst. The three trained algorithms are much better than Random, but none of them reaches Zero Torque. Each value is the average of three different trainings, so it gives a better idea of how the algorithm really works and not just of one good run.



This ranking only tells us which method gets the highest reward, but not which one controls the arm better. The reward penalises the distance to the target and the control effort together, so a policy that barely moves can get a good score.

7. Training Stability

Expected SARSA gets the highest average reward but changes the most between trainings, while Q-Learning is the most consistent but the lowest. To see how much the result depends on the training seed, we trained each algorithm three times for 200,000 steps (seeds 42, 123 and 2026) with its best Optuna hyperparameters. All the policies were evaluated on the same 50 episodes (seeds 5000–5049), so the differences between runs come from the training and not from the test episodes.

Algorithm	Seed 42	Seed 123	Seed 2026	Mean	Std. across runs
Q-Learning	−22.50	−20.97	−21.94	−21.80	0.77
SARSA	−20.01	−19.49	−22.74	−20.75	1.75
Expected SARSA	−21.10	−14.37	−15.19	−16.89	3.67

Std. → Standard deviation

With seed 42, Expected SARSA (−21.10) is not better than SARSA (−20.01) or Q-Learning (−22.50). With seeds 123 and 2026 it gets a much better result (−14.37 and −15.19). So its good average comes from two of its three trainings, and that is why its standard deviation (3.67) is much higher than the one of Q-Learning (0.77). SARSA is in the middle (1.75). In other words, there is a trade-off between getting a good average and being consistent.

Zero-torque usage is different between runs and algorithms:

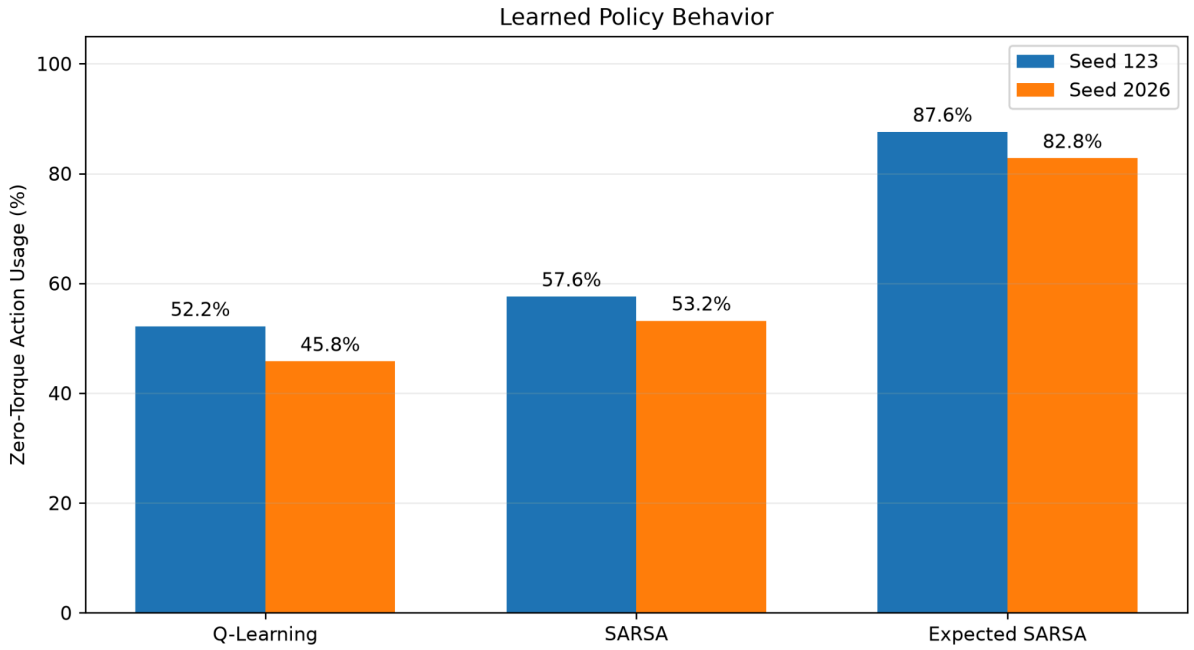
Algorithm	Seed 42	Seed 123	Seed 2026
Q-Learning	47%	52.2%	45.8%
SARSA	55.5%	57.6%	53.2%
Expected SARSA	57.3%	87.6%	82.8%

Expected SARSA chooses zero force in 87.6% and 82.8% of its decisions in the seed 123 and 2026 runs, which are also its two best runs by reward. This agrees with the idea that part of its advantage comes from moving less and not from reaching the target more accurately (Section 9).

Three seeds are enough to describe tendencies, but not to say that the differences between algorithms are statistically significant. In addition, we looked at the evaluation seeds 5000–5049 several times during the analysis, so we consider them a common evaluation set and not a completely independent test set.

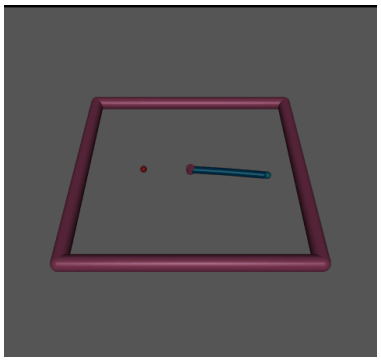
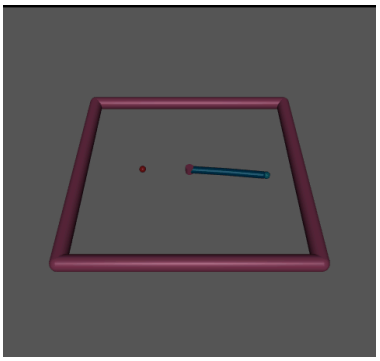
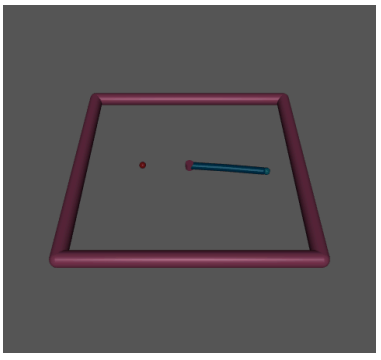
8. Learned Policy Behavior

The policies are very different in how often they apply no force. In the seed 2026 evaluation, Q-Learning chose zero torque in 45.8% of its decisions and SARSA in 53.2%, so about half of the time. Expected SARSA chose it in 82.8% of the decisions, and in 87.6% with seed 123.



The figure shows that Q-Learning and SARSA use zero force about half of the time in both seeds (123 and 2026), while Expected SARSA uses it much more often. This difference is a clear indication of the fact that Expected SARSA behaves in a different way from the other two algorithms.

Furthermore, we also recorded one evaluation episode of each greedy policy as a GIF (models trained with seed 2026, evaluation seed 5000). It is only one episode, so it shows an example of the behaviour and not a general rule.

Q-Learning	SARSA	Expected SARSA
		

- **Q-Learning**: The arm moves actively during the episode. It bends several times and ends in a "V" shape, with the fingertip near the target area. It does not stay still, but it does not stop exactly on the target either.
- **SARSA**: The arm also moves a lot. It starts stretched out to the right, then folds and ends close to its base, with the fingertip clearly nearer to the target than at the beginning. The movement is active and not very smooth.
- **Expected SARSA**: The arm practically does not move. It **stays in its initial position** during the whole episode, which is very similar to the Zero Torque baseline and agrees with its high percentage of zero-force actions.

We also measured the distance to the target at each step (seed 123 models, same 50 evaluation episodes). Zero Torque ends at 0.230 from the target. Q-Learning and SARSA end at 0.154 and are closer than Zero Torque in 34 and 30 of the 50 episodes. Expected SARSA ends at 0.231, almost the same as Zero Torque, and it is closer in only 4 episodes. So the Expected SARSA policy of seed 123 mostly imitates Zero Torque: its good reward (−14.37) comes from avoiding the control cost, not from reaching the target. Q-Learning and SARSA do move towards the target, but the force they use makes their reward lower.

9. Limitations and Discussion

In this section we discuss what we did not manage to achieve and why this may have happened. To be honest, our main result is not what we expected: we did not find a policy that is better than Zero Torque, the policy that never applies force. The three algorithms are clearly better than Random, but the agents seem to have learned mainly to use less force, not to reach the target. We can see this most clearly in Expected SARSA. For Q-Learning and SARSA the distance analysis with seed 123 shows that they do get closer to the target than Zero Torque, but they pay a higher control cost.

Why Zero Torque is hard to beat

In every step, the agent gets a negative reward for being far from the target and another one for using force. Zero Torque never uses force, so it has no control cost, and it gets −11.48 in 50 steps. We also tested the 16 constant actions in a pilot test (50 episodes, seeds 1000–1049). The zero action (0, 0) got −11.68, the best of all, while the next best constant actions got about −19.7 and the worst −106.9. This means that **a learned policy has to reduce the distance more than the control cost it adds**, and that is difficult for a table with so little data.

Limitations of tabular learning here

- **Sparse coverage:** The discretisation has 3^10 = 59,049 states and 16 actions, so the Q-table is very large. With only 200,000 training steps, the agent

cannot visit most state–action pairs many times. Many entries are never updated, and in the evaluation the ties between equal Q-values are broken randomly.

- **Coarse observations:** With three bins per variable, the agent cannot see small changes in the position of the arm or the distance to the target.
- **Action grid:** Four bins over [−1, 1] give joint torques of −1, −0.5, 0 and 0.5, so the grid is not symmetric and it cannot apply +1. With three bins the values would be −1, −1/3 and 1/3, with no zero action.
- **No generalisation:** A table treats close states as completely different, so what the agent learns in one state does not help in a similar state.

Overall, our results show that all three algorithms are clearly better than Random. Expected SARSA has the best average reward, but it is also the most variable between trainings, and its best runs use zero force in more than 80% of the decisions. However, we have only three training seeds and we used the same evaluation set many times, so the ranking between algorithms is a tendency and not a statistically proven difference. In addition, the reward alone cannot tell us whether an agent gets closer to the target, and we measured the distance only for the seed 123 models.

If we had more time, we would improve the analysis in several ways. First, we measured the distance to the target only for the seed 123 models, so we would record it for all the runs and report a success rate, for example the percentage of episodes where the arm ends close enough to the target. The environment already gives *reward_dist* and *reward_ctrl* in *info*, so this would be easy to do.
Second, we would train with more seeds, because three seeds are not enough to be sure about the differences between algorithms. Finally, we would try a finer or non-uniform discretisation, or a method that can generalise between similar states, because the tabular approach treats every state as completely different.

10. Individual Contributions

Both students worked together throughout the practical part of the assignment, including the implementation of the code, the experiments with Optuna, the training of the algorithms, and the analysis of the results. For example, we found out that expected sarsa was not moving while watching the GIFs. We discussed the decisions and solved the problems together.
For the report, we divided the writing tasks, **Iria** was responsible for the environment, methodology and Optuna analysis, while **Lucia** focused on the longer training, TensorBoard, and final evaluation. Finally, we wrote the conclusions together and reviewed the complete report to make sure everything was consistent.